

Trip Expense Tracker

Backend Design Document (v2) | FastAPI + SQLAlchemy + SQLite

1. Overview

A backend service supporting a group trip expense tracker. Multiple friends on a trip log expenses against three categories, each with a shared daily-per-person spending limit. The backend owns all aggregation and calculation logic (spend totals, limit comparisons, and payer suggestions) so that every client renders pre-computed, consistent figures rather than deriving them independently. The system does not implement authentication in this version; a lightweight, locally-stored user identity (device-generated name entry) is used instead, since the application is intended for use among trusted friends.

The backend follows a layered architecture: **routers** (HTTP concerns, request/response schemas) → **services** (business logic, validation, orchestration) → **repositories** (database access) → **models** (SQLModel table definitions). Domain-level exceptions are raised in the service layer and translated to HTTP responses only at the router boundary via a single centralized exception handler.

2. Database Schema

Five tables make up the schema. **TripUser** is a many-to-many join table linking users to the trips they've joined. **TripLimit** stores one row per spend category per trip, representing a per-day, per-person allowance that applies uniformly to every trip member in this version.

2.1 User

Column	Type	Notes
<code>id</code>	UUID (PK)	Generated server-side on creation
<code>name</code>	string	Locally entered name; not unique, not authenticated
<code>created_at</code>	datetime	Defaults to time of creation

2.2 Trip

Column	Type	Notes
<code>id</code>	UUID (PK)	Generated server-side on creation
<code>name</code>	string	Trip display name
<code>start_date</code>	date	Used to compute days elapsed for all-time summaries
<code>created_at</code>	datetime	Defaults to time of creation

2.3 TripUser (join table)

Column	Type	Notes
<code>trip_id</code>	UUID (PK, FK -> Trip.id)	Composite primary key with user_id

<code>user_id</code>	UUID (PK, FK -> User.id)	Composite primary key with trip_id
<code>joined_at</code>	datetime	Defaults to time of join

2.4 TripLimit

Column	Type	Notes
<code>id</code>	UUID (PK)	Generated server-side on creation
<code>trip_id</code>	UUID (FK -> Trip.id)	One row per category per trip
<code>category</code>	enum	food / cabs / incidental
<code>daily_limit_amount</code>	float	Per-day, per-person allowance; uniform across all trip members (v1). Defaults to 0 for any category not explicitly set at trip creation.

2.5 Expense

Column	Type	Notes
<code>id</code>	UUID (PK)	Generated server-side on creation
<code>trip_id</code>	UUID (FK -> Trip.id)	Trip this expense belongs to
<code>user_id</code>	UUID (FK -> User.id)	The user who paid
<code>category</code>	enum	food / cabs / incidental
<code>payment_type</code>	enum	cash / credit_card / forex_card
<code>amount</code>	float	
<code>description</code>	string, nullable	Optional free-text note
<code>timestamp</code>	datetime	Defaults to "now" if omitted; explicit value allowed for backdated entries

3. API Endpoints

All endpoints are prefixed at the application root (no versioning prefix in v1). Request and response bodies are validated via Pydantic schemas. All calculation/aggregation logic executes server-side.

Method	Path	Description
POST	/users	Register a local user identity; returns a user_id for the client to persist.
GET	/trips?user_id=	List all trips a given user has joined (lightweight: id, name, start_date). Used to populate the frontend home screen / trip picker.
POST	/trips	Create a trip with category limits; any subset of categories may be supplied, unspecified categories default to a daily_limit_amount of 0. Creator is auto-joined as the first member.
GET	/trips/{trip_id}	Fetch full trip details: metadata, limits, and current member list.
PUT	/trips/{trip_id}/users/{user_id}	Join a trip (idempotent — safe to retry).
PUT	/trips/{trip_id}/limits	Partially update category limits for a trip — only the categories included in the request body are changed; other existing category limits are left untouched.
POST	/trips/{trip_id}/expenses	Log a new expense against a trip.
GET	/trips/{trip_id}/expenses	List expenses for a trip; optional filters: user_id, category, day.
GET	/trips/{trip_id}/summary	Aggregated per-user, per-category spend vs. limit, and per-payment-type breakdown. Optional ?day= filter; omitted = all-time (cumulative vs. limit × days elapsed).
POST	/trips/{trip_id}/suggest-payer	Suggests which trip member should pay for a given category and day (optionally a specific amount), ranked by remaining headroom (that day's category limit minus that user's spend on that day/category).

3.1 Key Business Rules

- Limits are **per day, per person**, uniform across all trip members in v1.
- Trip creation and limit updates both accept a **partial** set of categories — any category not included defaults to (on creation) or remains at (on update) its existing value; unset categories default to 0 on creation.
- Day-filtered summaries compare that day's spend directly against the daily limit.
- All-time summaries compare cumulative spend against *daily_limit × days elapsed since trip.start_date*.
- The suggest-payer algorithm evaluates headroom for a **caller-specified day** (*that day's category limit minus that user's spend in that category on that day*) — the client decides which day to request a suggestion for.
- Joining a trip and setting limits are both idempotent / safe to retry with no unintended side effects.

4. Requirements — Negative Scenario Handling

The service layer raises domain-level exceptions (decoupled from HTTP) which are translated to HTTP responses by a single centralized exception handler registered on the base `DomainException` class. The table below enumerates every handled negative scenario, its triggering exception, and the resulting HTTP status code.

Scenario	Exception	Status	Notes
Trip ID does not exist	<code>TripNotFoundEr or</code>	404	Raised on any endpoint that references a <code>trip_id</code> path parameter for a trip that was never created.
User ID does not exist	<code>UserNotFoundEr or</code>	404	Raised when creating a trip with an unknown <code>creator_user_id</code> , joining a trip as an unknown user, logging an expense as an unknown user, or listing trips for an unknown <code>user_id</code> .
Expense logged by a user who has not joined the trip	<code>UserNotInTripEr ror</code>	403	A registered user must be a trip member (via join) before they can log an expense against that trip.
Summary or suggest-payer requested before limits are set	<code>TripLimitsNotSe tError</code>	409	Trip exists but has no <code>TripLimit</code> rows (or is missing the specific category being queried) — treated as a conflict, not a 404, since the trip itself is valid.
Trip limits payload is empty, contains an unrecognized category, or lists the same category twice	<code>InvalidTripExpe nseCategoryErro r</code>	400	Enforced on both trip creation and limit updates. Unlike v1, the payload no longer needs to cover all three categories — it must simply be a non-empty set of valid, non-duplicated categories.
Duplicate join request for a trip the user already belongs to	<code>None — handled gracefully</code>	200	<code>PUT /trips/{trip_id}/users/{user_id}</code> is idempotent: a repeat call returns the existing membership rather than erroring or creating a duplicate row.
Repeated or partial limit-setting requests	<code>None — handled gracefully</code>	200	<code>PUT /trips/{trip_id}/limits</code> updates only the categories supplied, in one transaction; safe to call multiple times, including with a single category at a time.
Suggest-payer with no eligible candidate	<code>None — signalled in response body</code>	200	When a requested amount would exceed every member's remaining headroom for the specified day, <code>suggested_user_id</code> is returned as null and all candidates are marked ineligible, rather than raising an error.
Malformed request body / wrong field types	<code>Pydantic ValidationError (framework-level)</code>	422	Handled automatically by FastAPI's request validation layer before reaching the router function.

4.1 Exception-to-Status Mapping (Summary)

Exception	HTTP Status
<code>TripNotFoundEr or</code>	404 Not Found
<code>UserNotFoundEr or</code>	404 Not Found
<code>TripLimitsNotSetError</code>	409 Conflict

UserNotInTripError	403 Forbidden
InvalidTripExpenseCategoryError	400 Bad Request